

Software Engineering Lab Experiments

Laboratory Manual

Name : THONDURU SUSHMA

Register Number: 192325135

Department : AIML

Course : CSA1011

Faculty : Dr. Rami Reddy

SAVEETHA SCHOOL OF ENGINEERING

Experiment No. 1

Create a Kanban Board to Visualize the Tasks using Jira Software

Aim

The objective of this experiment is to create a Kanban Board using Jira Software, add sample tasks, and simulate a software development workflow by moving tasks through different stages such as To Do, In Progress, and Done.

Software Requirements

Software	Version
Jira Cloud	Web Application
Web Browser	Google Chrome / Microsoft Edge
Internet Connection	Required

Theory

Jira is a popular project management and issue tracking tool used by software development teams to plan, monitor, and manage projects.

A Kanban Board is a visual workflow board that organizes work into columns representing different stages of development. Team members move tasks from one column to another as work progresses. Kanban improves visibility, reduces bottlenecks, and helps teams focus on ongoing work.

In software engineering, Kanban is widely used because it supports continuous delivery, improves collaboration, simplifies task tracking, and enables project managers to monitor progress in real time. It is easy to understand and suitable for agile development practices.

Procedure

Step 1

Open a web browser on your system.

Navigate to the Jira Cloud website by entering its URL in the address bar.

Sign in using your registered Jira account credentials.

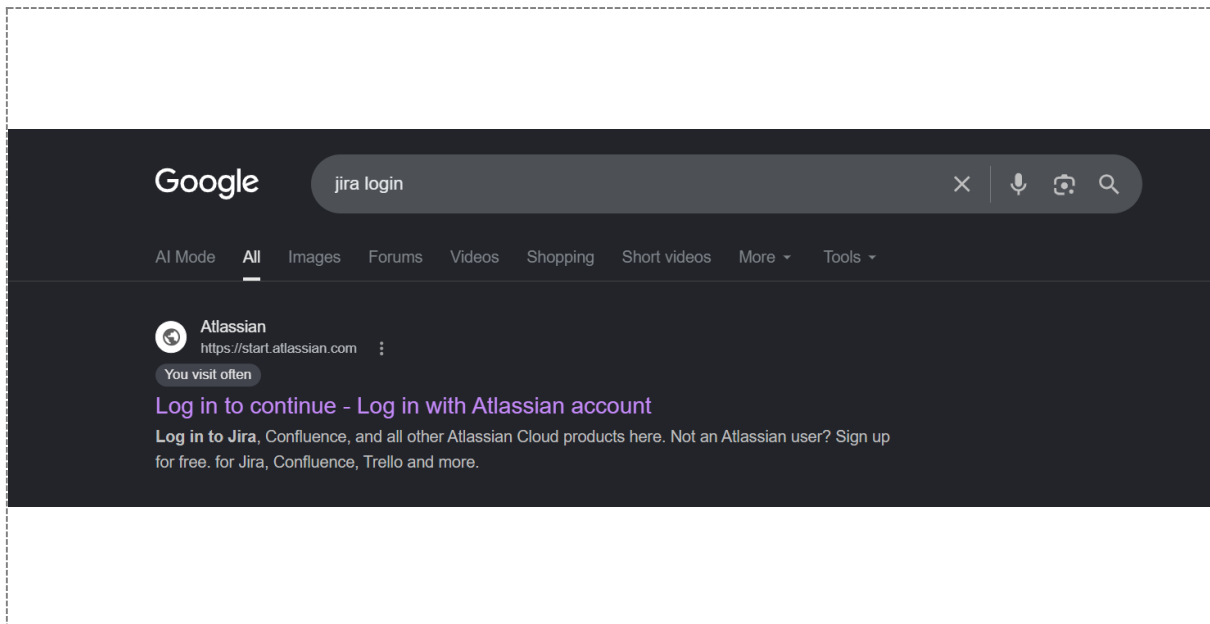


Figure 1. Opening Jira Cloud.

Step 2

After logging in successfully, click the "Jira" option available in the left navigation panel.

This action opens the main Jira workspace where projects and boards can be managed.

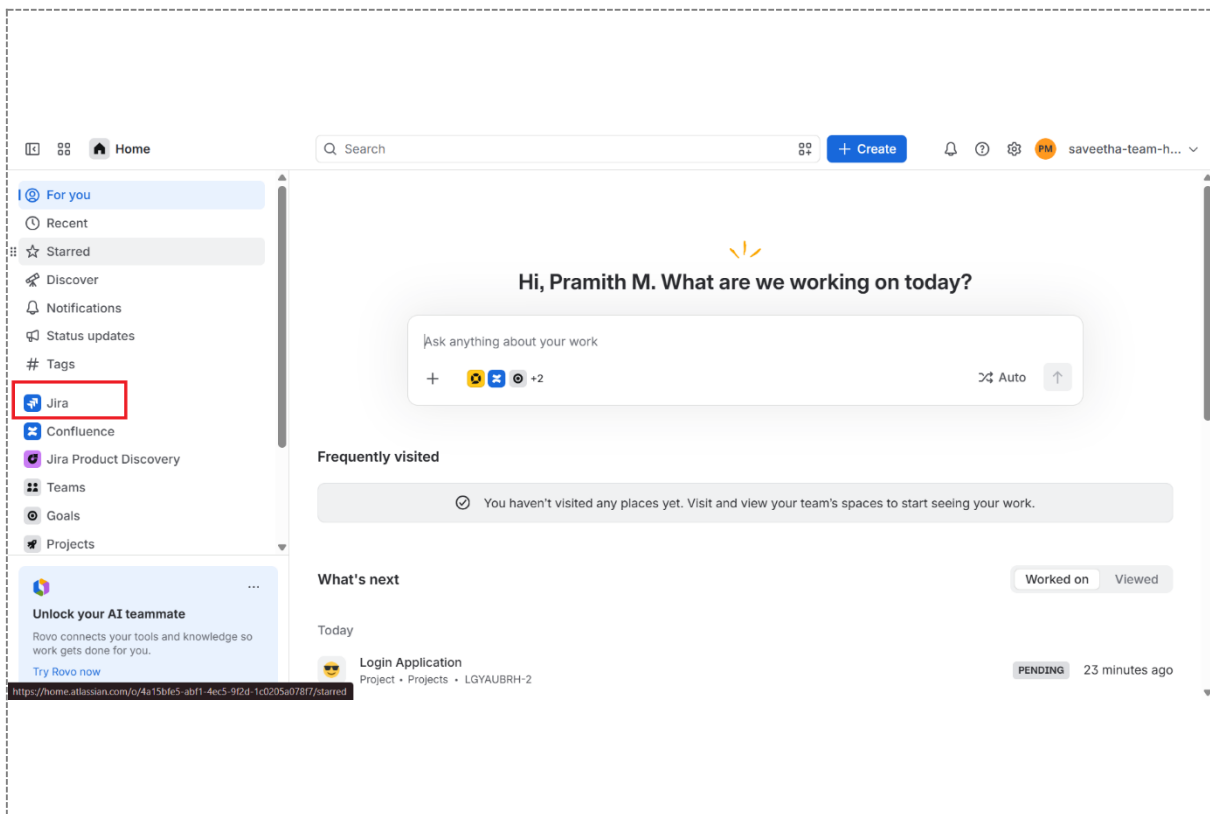


Figure 2. Opening Jira Workspace.

Step 3

Click the "Create Space" button located within the Jira workspace to begin creating a new project space.

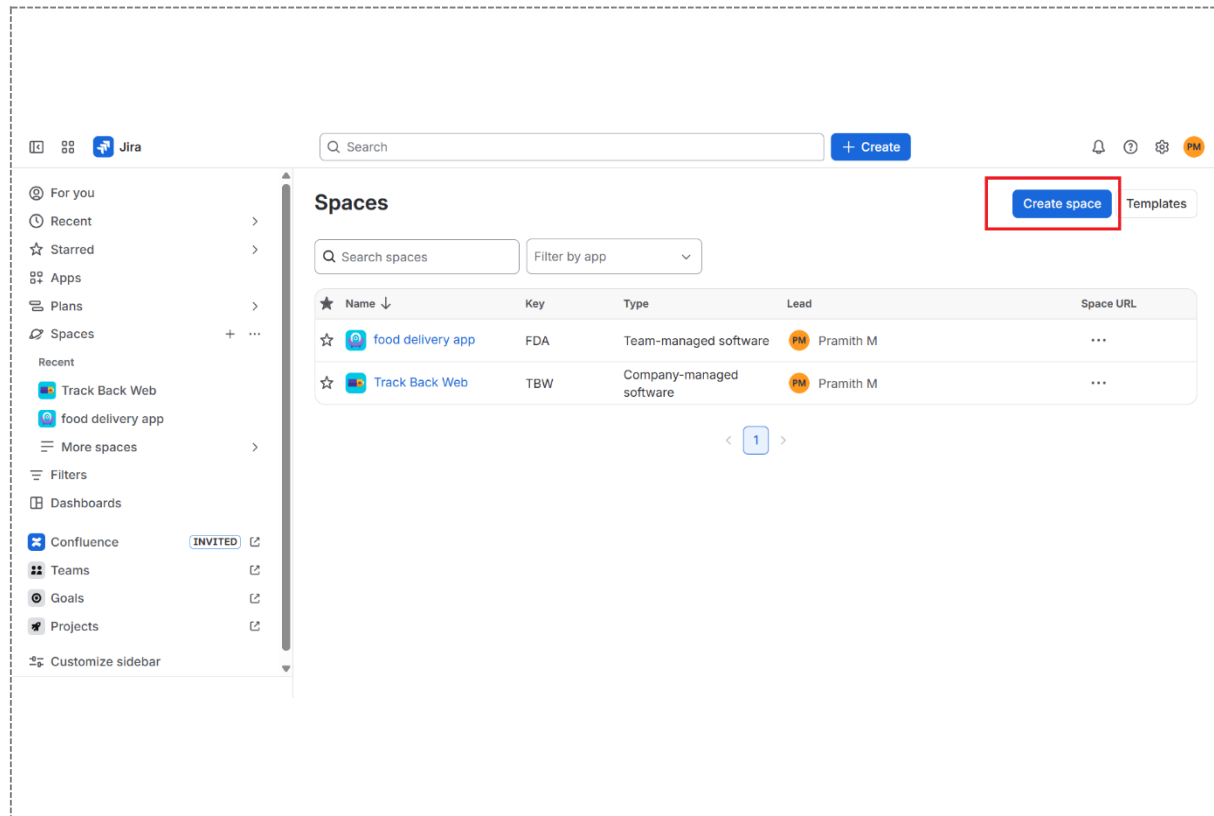


Figure 3. Creating a New Space.

Step 4

Under the Category option, select Software Development.

Select the Kanban Template from the list of available templates.

Click the Next button to proceed.

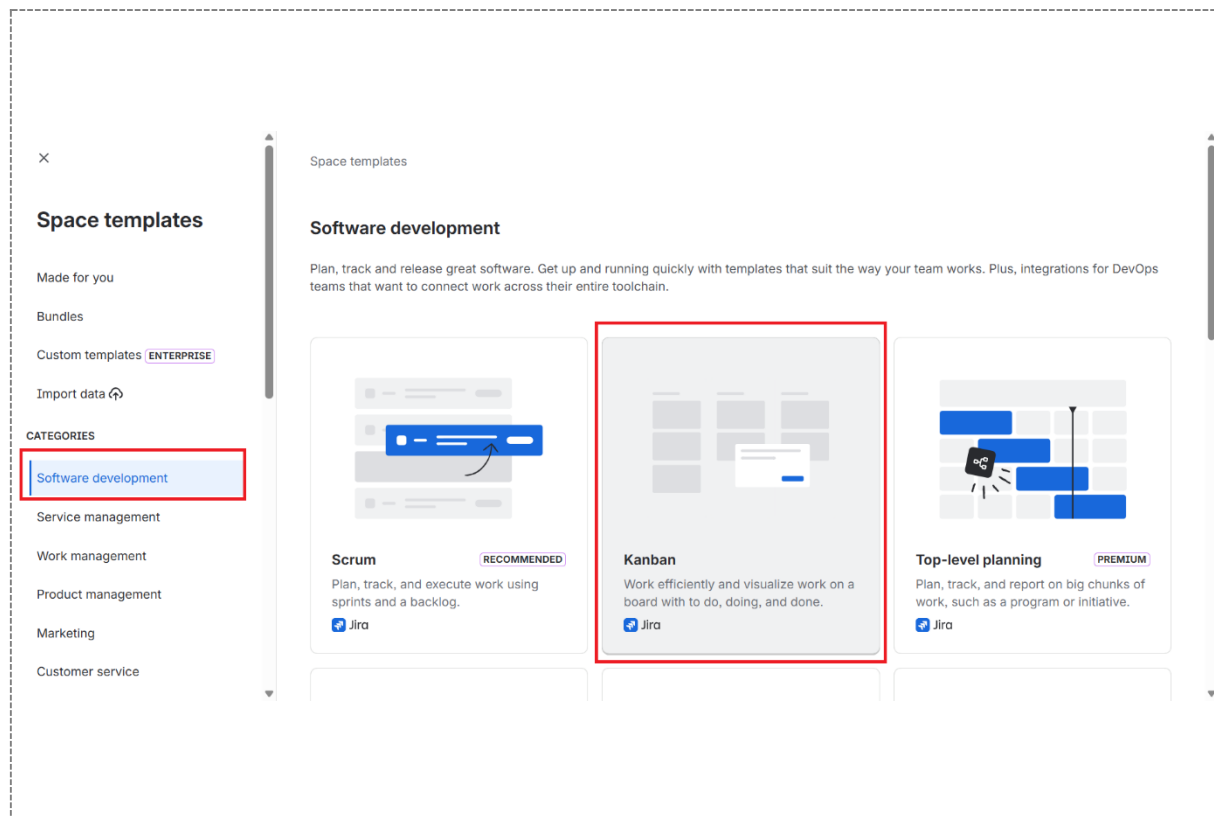


Figure 4. Selecting the Kanban Template.

Step 5

Enter the project Name as "Student Task Management System".

Choose the Management Type as Company-managed.

Enter the project Key as "STMS".

Click the Next button to complete the space setup.

Screenshot Placeholder

← Back to templates

Name your space

Required fields are marked with an asterisk *

Name *

How your space is managed *

 Company-managed



[Learn more about space types](#)

Space permissions

Customize project settings

Customize and share settings with other projects.

[Try it free for 14 days](#) 

Key * 

Step 1 of 3

Next

Figure 5. Entering Space Details.

Step 6

The Kanban Board is created successfully and displayed on the screen.

The board contains the following workflow columns:

- Backlog
- Selected for Development

- In Progress
- Done

The Backlog column holds tasks that are yet to be started. Selected for Development contains tasks chosen for the current cycle. In Progress represents tasks currently being worked on, and Done contains completed tasks.

Screenshot Placeholder

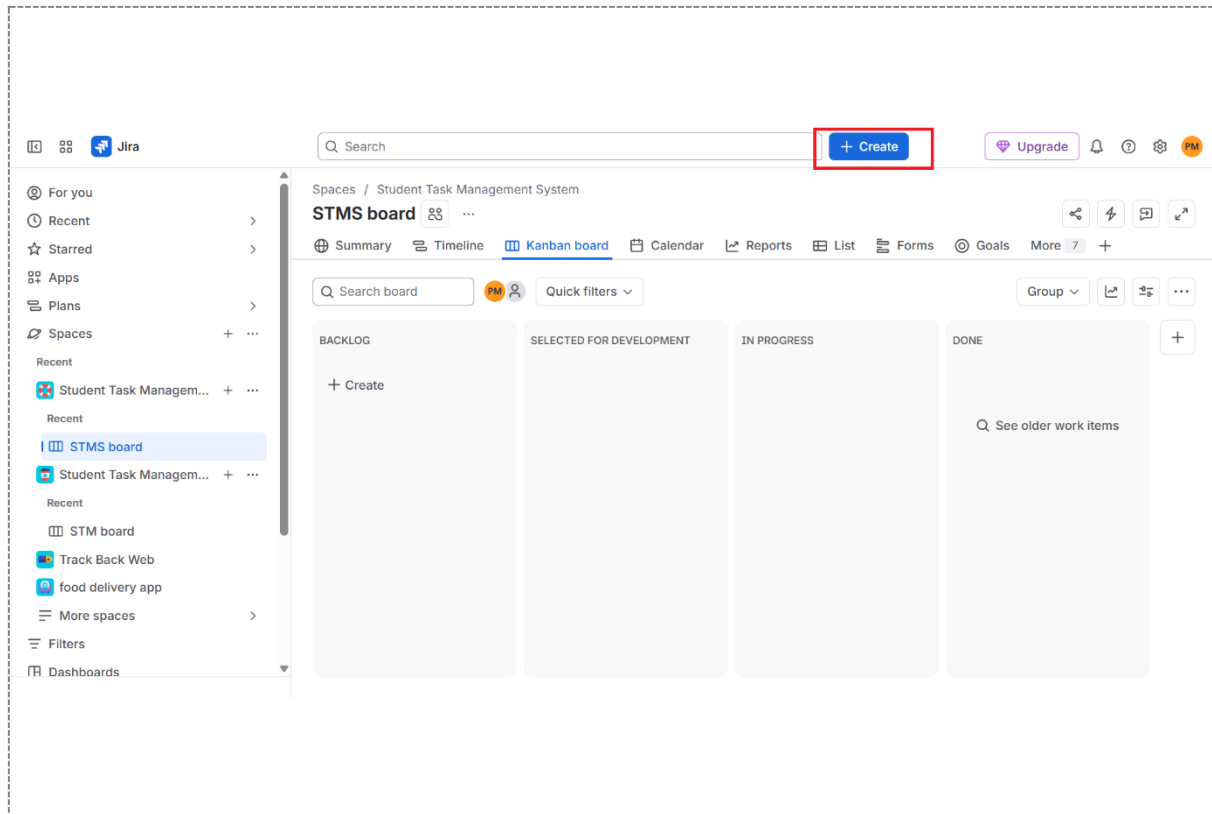


Figure 6. Newly Created Kanban Board.

Step 7

Click the Create button to add new tasks to the board.

Add the following five sample tasks as listed in the table below.

Task ID	Task Name	Description
STM-1	Gather Project Requirements	Collect functional and non-functional requirements.
STM-2	Design Login Page	Design the login user interface.
STM-3	Develop Database	Create database tables and relationships.

Task ID	Task Name	Description
STM-4	Implement User Authentication	Develop login and registration functionality.
STM-5	Testing and Bug Fixing	Test the application and fix defects.

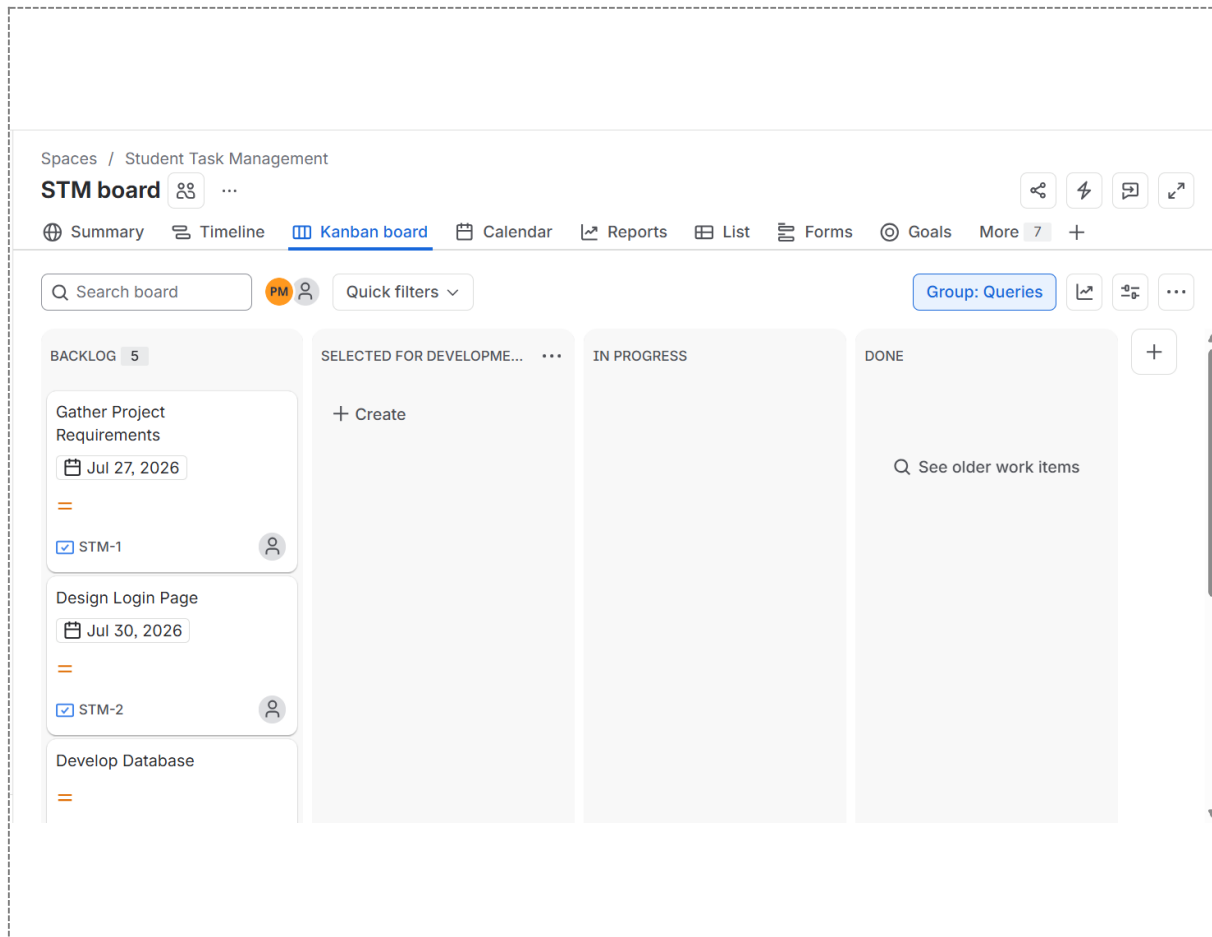


Figure 7. Creating Sample Tasks.

Step 8

Move the tasks across the different workflow stages by dragging and dropping them between columns, as shown below:

Gather Project Requirements: Backlog → Selected for Development → In Progress → Done

Design Login Page: Backlog → In Progress → Done

Develop Database: Backlog → In Progress

Testing and Bug Fixing: Selected for Development

Moving issues across the board represents the actual progress of tasks during the software development process, allowing the entire team to visually track project status at any point in time.

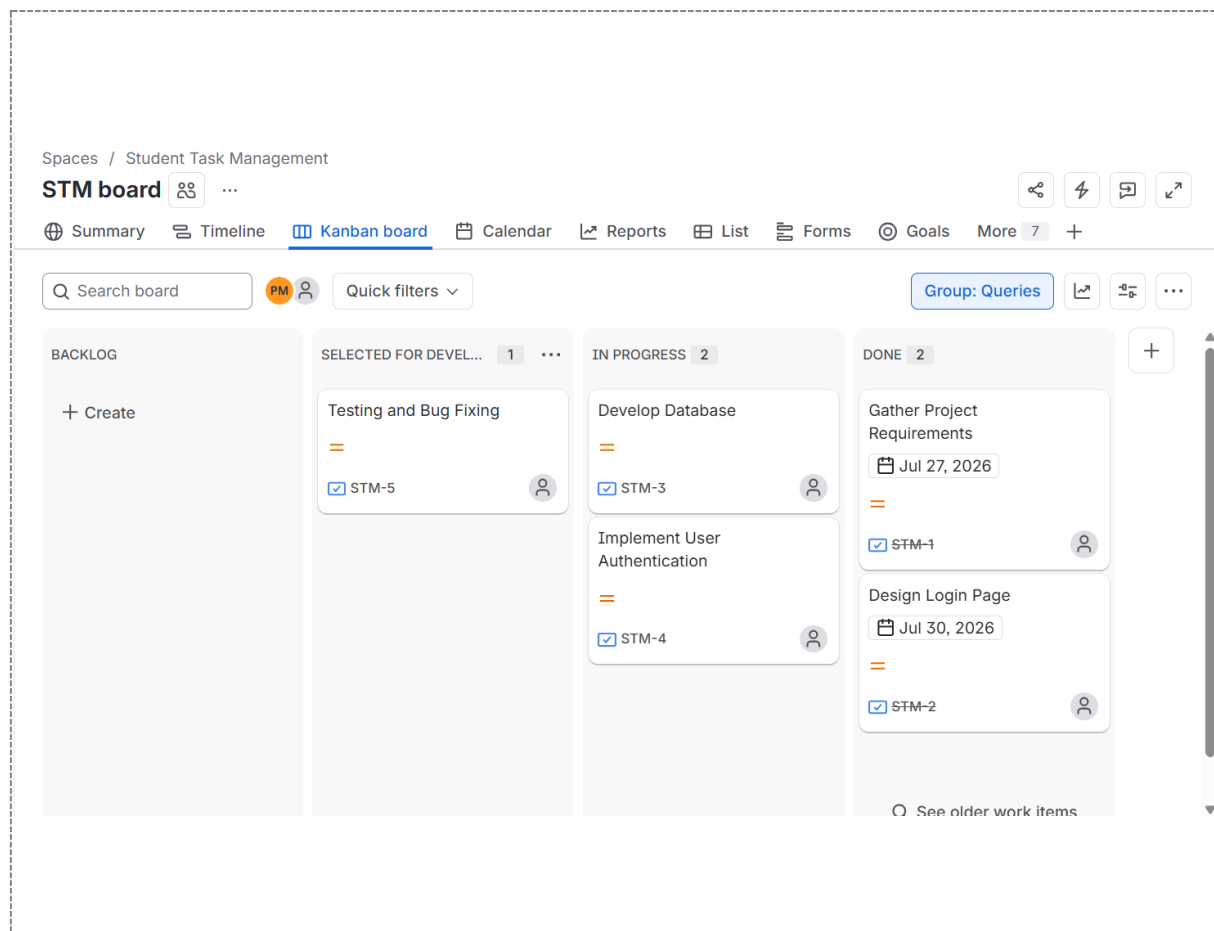


Figure 8. Moving Tasks Across Workflow Stages.

Observation

Task Name	Initial Status	Final Status
Gather Project Requirements	Backlog	Done
Design Login Page	Backlog	Done
Develop Database	Backlog	In Progress
Implement User Authentication	Backlog	Backlog
Testing and Bug Fixing	Backlog	Selected for Development

Result

The Kanban Board was successfully created using Jira Software. Five sample tasks were added and moved across different workflow stages to simulate a software development process. The experiment demonstrated how Jira helps teams visualize, organize, and track project progress efficiently.

Experiment No. 2

Sketch a Simple Prototype of a Bus Ticket Booking System using Figma Tool

Aim

To design a simple prototype of a Bus Ticket Booking System using the Figma tool by creating user interface screens and connecting them through prototype interactions.

Software Requirements

Software	Version
Figma	Web Application
Web Browser	Google Chrome / Microsoft Edge
Internet Connection	Required

Theory

Figma is a cloud-based user interface (UI) and user experience (UX) design tool widely used for designing websites, mobile applications, and software prototypes. It enables designers and developers to collaborate in real time, making the design process faster and more efficient.

A prototype is a visual representation of an application that demonstrates how users interact with different screens before the actual software is developed. Prototyping helps identify design improvements, reduces development errors, and improves the overall user experience.

In software engineering, Figma is commonly used to create wireframes, UI designs, and interactive prototypes. By connecting screens through clickable interactions, developers and stakeholders can visualize the application's workflow before implementation.

Procedure

Step 1: Open Figma

Open any web browser.

Navigate to the Figma website.

Sign in using your Figma account credentials.

After successful login, the Figma dashboard will be displayed.

Step 2: Create a New Design File

Click the New Design File button.

A blank design workspace will open.

Rename the file as Bus Ticket Booking System.

Step 3: Create Mobile Frames

Select the Frame Tool (F) from the toolbar.

Choose a mobile device frame such as Android Large or iPhone 14.

Create six separate mobile frames.

Arrange all frames horizontally for better organization.

Step 4: Design the Splash Screen

Create the first screen containing the following components:

- Application Logo
- Bus Image or Icon

- Application Title
- Welcome Message
- Get Started Button

This screen introduces the application to the user.

Step 5: Design the Login Screen

Create the login interface with the following components:

- Username Text Field
- Password Text Field
- Login Button
- Forgot Password Link
- Sign Up Link

This screen allows users to log into the application.

Step 6: Design the Home Screen

Design the main booking page containing:

- Source Location
- Destination Location
- Journey Date
- Number of Passengers
- Search Button

This screen enables users to search for available buses.

Step 7: Design the Search Bus Screen

Display a list of available buses. Each bus should contain:

- Bus Name

- Departure Time
- Ticket Price
- Book Now Button

This screen allows users to select a preferred bus.

Step 8: Design the Seat Selection Screen

Create a simple seat layout using square shapes. Include:

- Available Seats
- Selected Seat
- Confirm Booking Button

This screen enables users to select their desired seat before booking.

Step 9: Design the Booking Confirmation Screen

Create the final screen displaying booking information. Include:

- Booking Successful Message
- Passenger Name
- Bus Name
- Seat Number
- Journey Date
- Download Ticket Button
- Home Button

This screen confirms that the ticket has been successfully booked.

Step 10: Create Prototype Connections

Open the Prototype tab.

- Connect the Get Started button to the Login Screen.

- Connect the Login button to the Home Screen.
- Connect the Search button to the Search Bus Screen.
- Connect the Book Now button to the Seat Selection Screen.
- Connect the Confirm Booking button to the Booking Confirmation Screen.
- Connect the Home button back to the Home Screen.

These connections simulate the complete user workflow.

Step 11: Run the Prototype

Click the Present button.

- Test every button by clicking it.
- Verify that each screen navigates correctly.
- Ensure the complete booking process functions smoothly from the Splash Screen to the Booking Confirmation Screen.

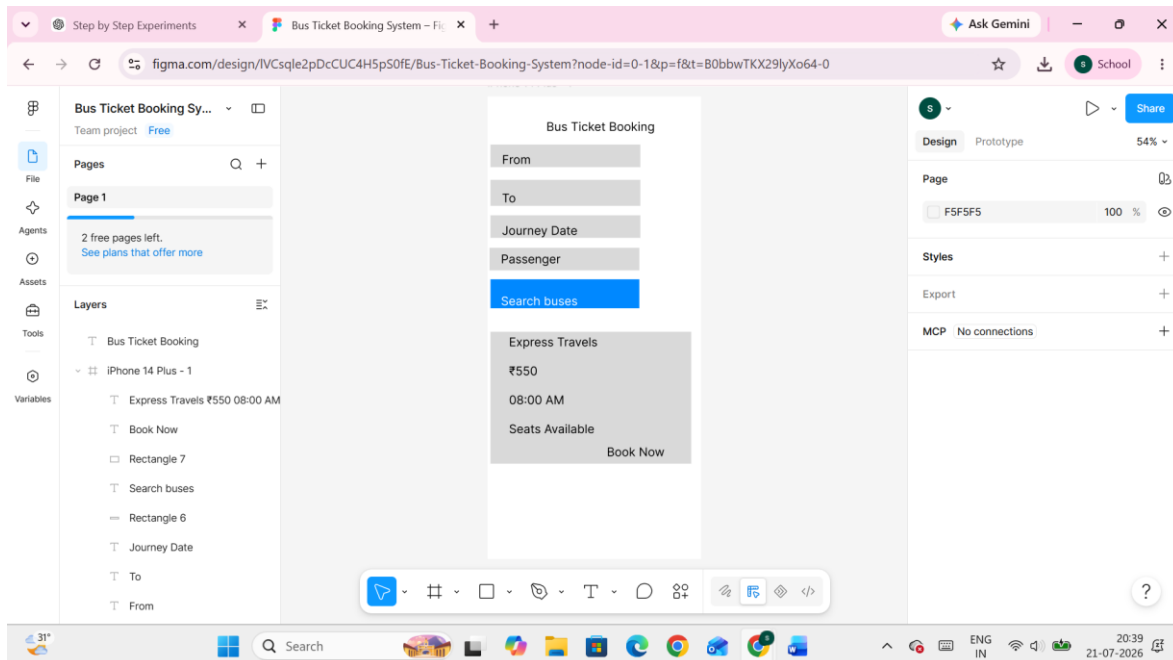
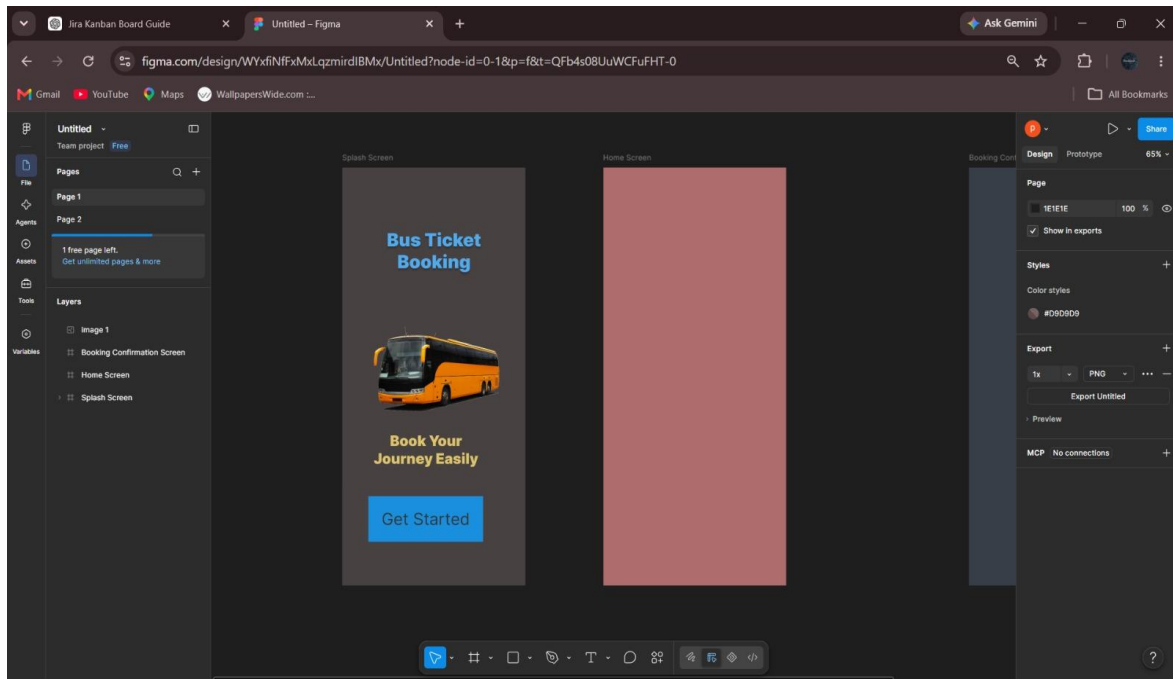
Observation

Screen Name	Components Designed	Prototype Link Status
Splash Screen	Logo, Title, Get Started Button	Linked to Login Screen
Login Screen	Username, Password, Login Button	Linked to Home Screen
Home Screen	Source, Destination, Date, Search Button	Linked to Search Bus Screen
Search Bus Screen	Bus List, Book Now Button	Linked to Seat Selection Screen
Seat Selection Screen	Seat Layout, Confirm Booking Button	Linked to Booking Confirmation Screen
Booking Confirmation Screen	Booking Details, Home Button	Linked back to Home Screen

Result

A simple prototype of the Bus Ticket Booking System was successfully designed using Figma. Six mobile screens were created and connected through prototype interactions to simulate the

complete user journey, from the Splash Screen to the Booking Confirmation Screen. The experiment demonstrated how Figma helps visualize an application's workflow before actual development begins.



Experiment No. 3

Design an E-Commerce Mobile Application Prototype using Figma

Aim

To design and connect a simple prototype of an E-Commerce Mobile Application named ShopEase using the Figma tool, demonstrating the complete user journey from the Welcome Screen to the Order Confirmation Screen.

Software Requirements

Software	Version
Figma	Web Application
Web Browser	Google Chrome / Microsoft Edge
Internet Connection	Required

Theory

Figma is a cloud-based UI/UX design tool that allows designers to create wireframes, screen designs, and interactive prototypes that can be shared and reviewed in real time. It is widely used to design mobile and web applications before development begins.

A clickable prototype connects individual screens using defined interactions, allowing stakeholders to experience the application's navigation flow as if it were a working product. This helps identify usability issues early and reduces costly changes later in development.

In an e-commerce application, prototyping the journey from browsing products to completing a purchase helps validate that the checkout flow is intuitive and that all essential screens, such as login, product details, and order confirmation, work together seamlessly.

Procedure

Step 1: Open Figma

1. Open a web browser.

2. Navigate to the Figma website.
3. Sign in using your Figma account.
4. Click New Design File to create a new project.

Step 2: Create a New Design File

1. Rename the design file as E-Commerce Mobile App Prototype.
2. Select the Frame Tool (F) from the toolbar.
3. Choose a mobile device frame (Android Large or iPhone 14).
4. Create six mobile frames for designing the application screens.

Step 3: Design the Welcome Screen

Create the first screen with the following components:

- Application Logo
- Application Name (ShopEase)
- Welcome Message
- Get Started Button

This screen serves as the entry point of the application.

Step 4: Design the Login Screen

Create the Login screen containing:

- Email Address field
- Password field
- Login Button
- Sign Up Link

This screen allows users to authenticate before accessing the application.

Step 5: Design the Home Screen

Design the Home screen with the following elements:

- Search Bar
- Product Categories
- Featured Products
- Bottom Navigation Bar

This screen provides users with easy access to products and categories.

Step 6: Design the Product Details Screen

Create a Product Details screen displaying:

- Product Image
- Product Name
- Product Price
- Product Description
- Add to Cart Button

This screen enables users to view detailed information about a selected product.

Step 7: Design the Shopping Cart Screen

Design the Shopping Cart screen containing:

- Selected Product List
- Quantity Selector
- Total Price
- Checkout Button

This screen allows users to review their selected products before placing an order.

Step 8: Design the Order Confirmation Screen

Create the final screen displaying:

- Order Successfully Placed message
- Order ID
- Thank You Message
- Continue Shopping Button

This screen confirms the successful completion of the purchase.

Step 9: Create Prototype Connections

Select the Prototype tab in Figma. Connect the screens using navigation links as follows:

- Welcome Screen → Login Screen
- Login Screen → Home Screen
- Home Screen → Product Details Screen
- Product Details Screen → Shopping Cart Screen
- Shopping Cart Screen → Order Confirmation Screen

Set the interaction type to On Click → Navigate To for each button.

Step 10: Preview the Prototype

1. Click the Present button.
2. Navigate through the prototype by clicking the buttons.
3. Verify that all screens are connected correctly and the navigation works as expected.

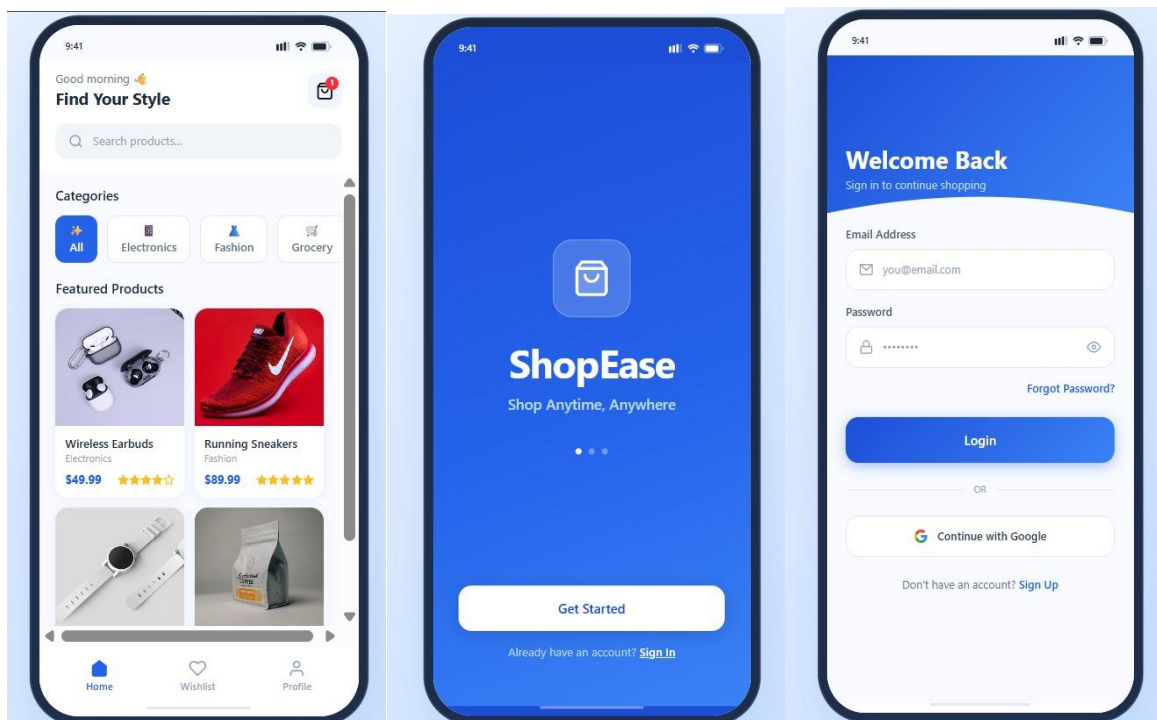
Observation

Screen Name	Key Components	Connected To
Welcome Screen	Logo, App Name, Get Started Button	Login Screen
Login Screen	Email, Password, Login Button	Home Screen
Home Screen	Search Bar, Categories, Featured Products	Product Details Screen

Screen Name	Key Components	Connected To
Product Details Screen	Product Info, Add to Cart Button	Shopping Cart Screen
Shopping Cart Screen	Product List, Total Price, Checkout Button	Order Confirmation Screen
Order Confirmation Screen	Order ID, Thank You Message	—

Result

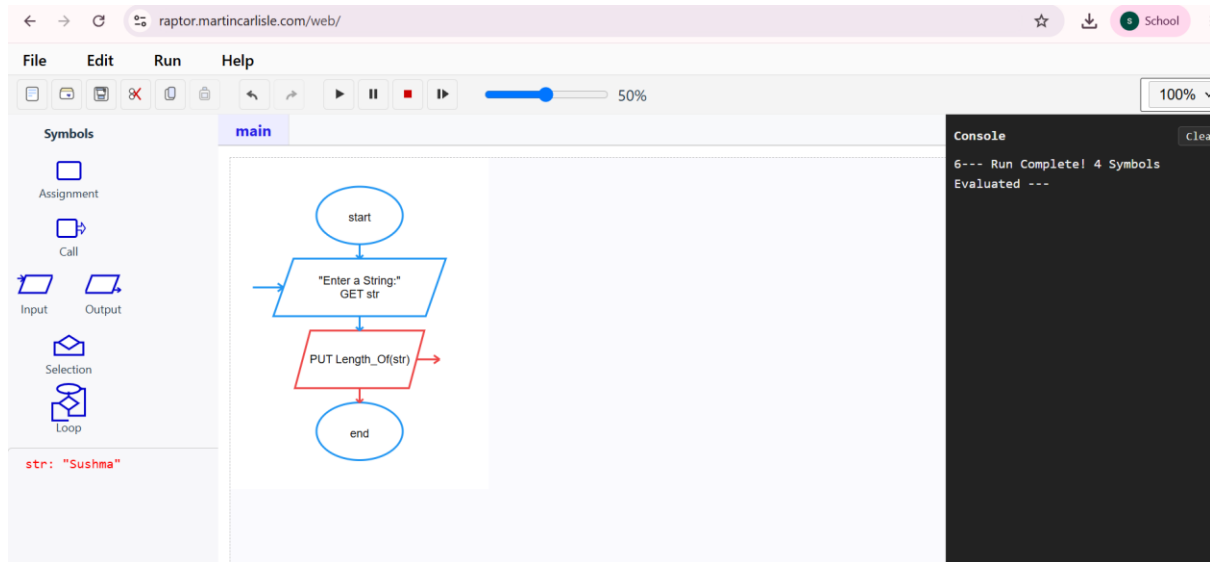
A prototype of the E-Commerce Mobile Application was successfully designed using the Figma tool. The prototype demonstrated the complete user flow from the Welcome Screen to the Order Confirmation Screen. The design was presented to stakeholders to gather feedback and obtain approval before proceeding with the application development phase.



Experiment No. 4

The string is a sequence of characters placed in double quotes (" "). Performing different operations on string data is called String Handling. Strings are immutable. Whenever a

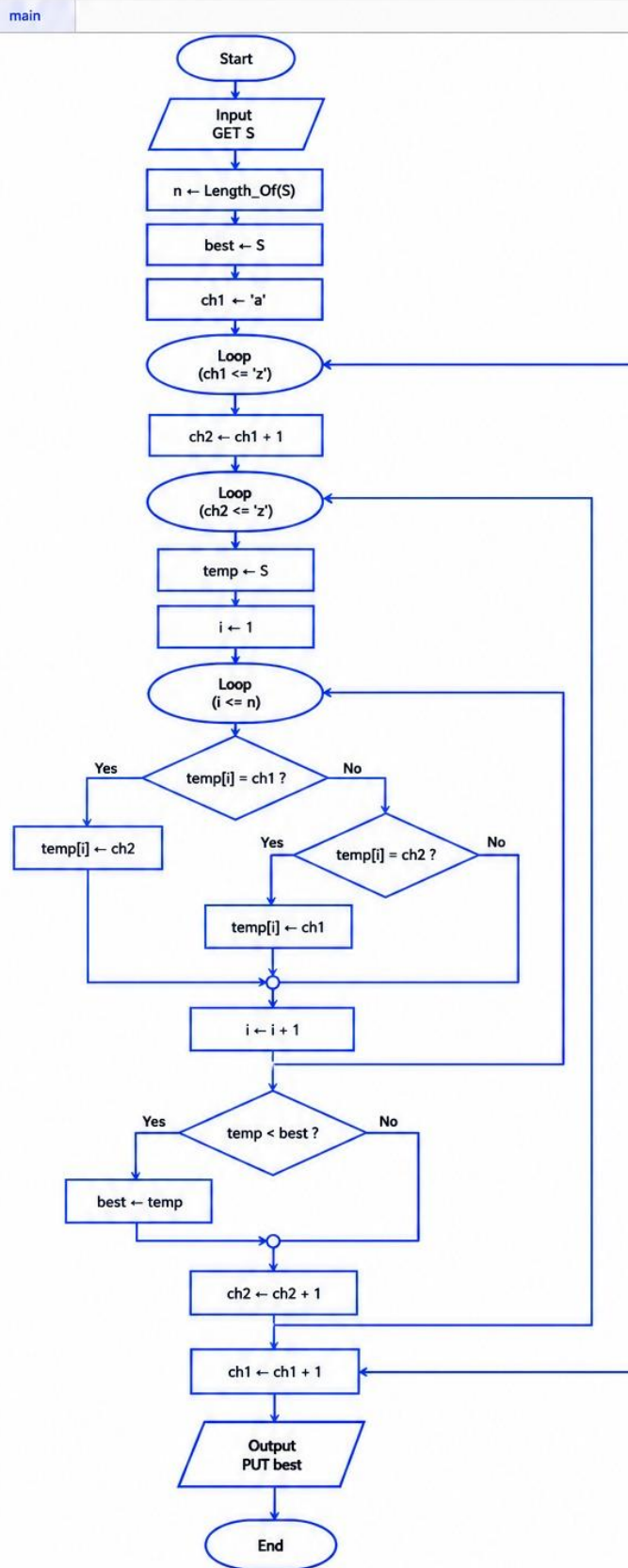
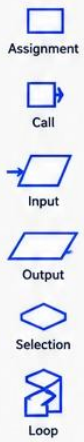
change to a String is made, an entirely new String is created. If we want to store a group of characters we can use a char array. String provides various methods to perform different operations on strings. Using Raptor draw a flowchart to display the total number of characters in the string and returns it.



Experiment No. 05

Using Raptor – Draw and validate a flowchart for a given string of lower case English alphabets. One can choose any two characters in the string and replace all the occurrences of the first character with the second character and replace all the occurrences of the second character with the first character. Using Raptor draw and validate the flowchart lexicographically smallest string that can be obtained by doing this operation at most once.

Symbols



Console

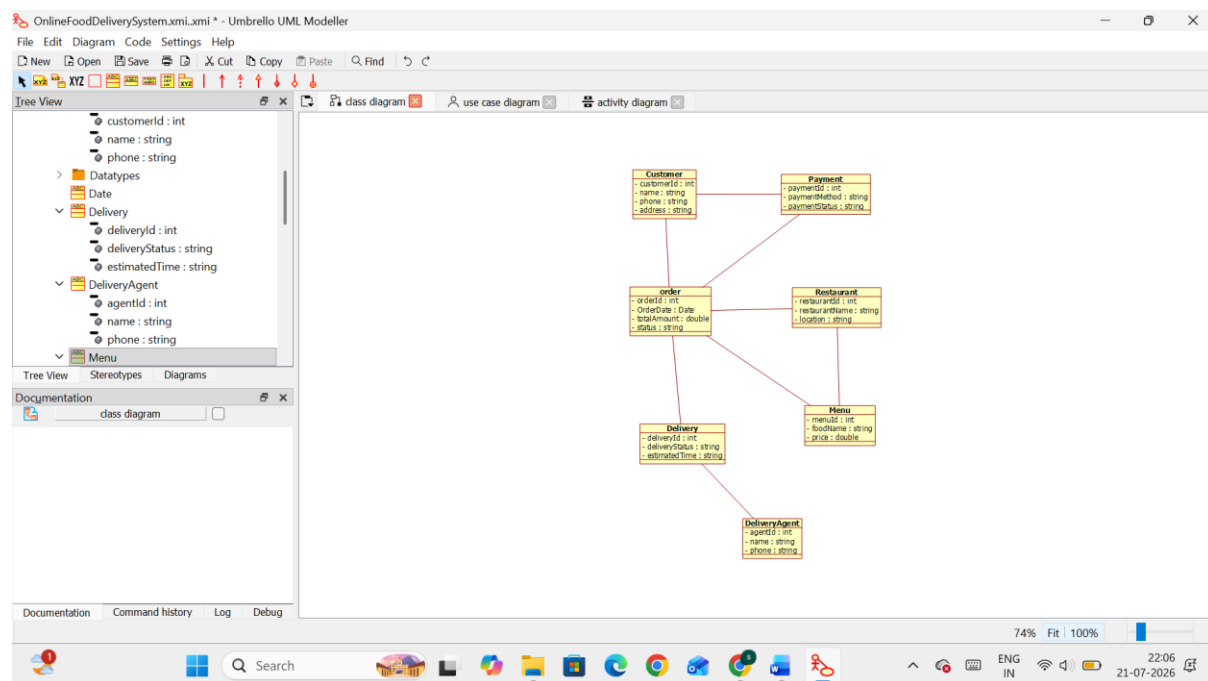
Clear

```

--- Run ---
Input S: hacking
abcking
Program finished.
  
```

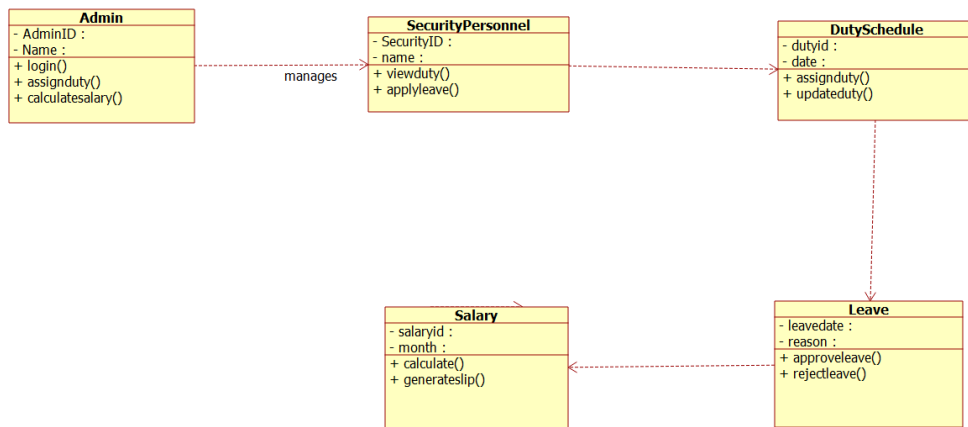
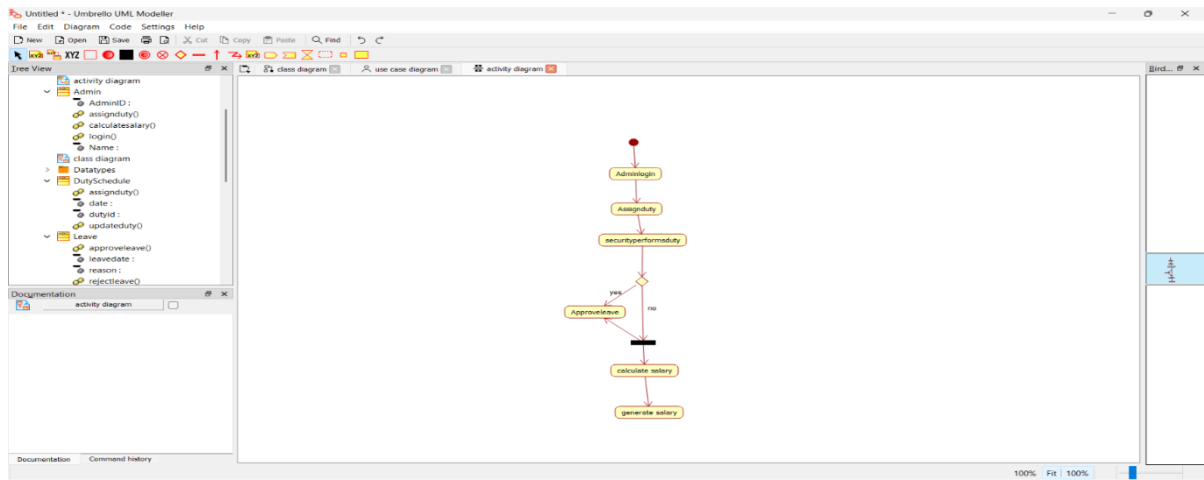
Experiment No. 6

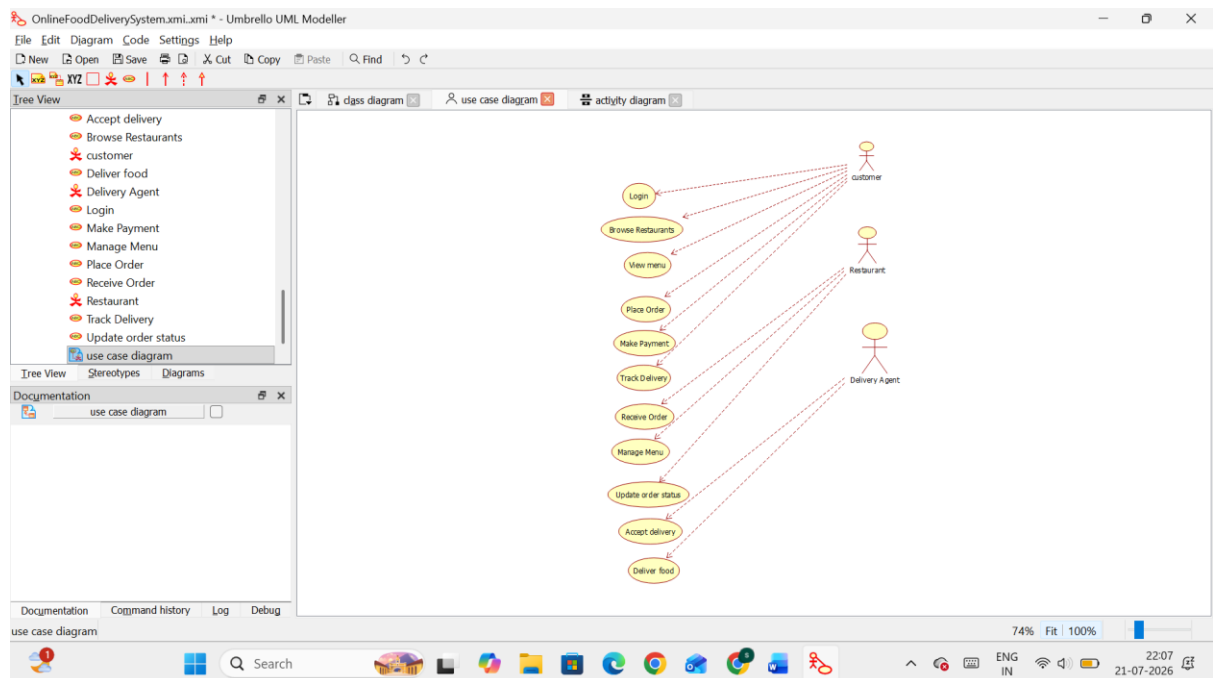
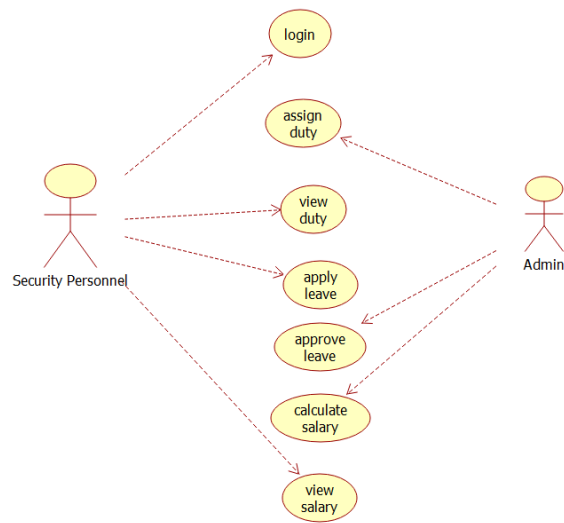
An online food delivery platform enables customers to browse restaurants, view menus, place food orders, make online payments, and track delivery status. Restaurants use the system to manage their menus and update order status, while delivery agents are responsible for accepting delivery requests and updating delivery progress. Design a Class Diagram, Use case Diagram and Activity Diagram for the Online Food Delivery System showing the relevant classes, their attributes, methods, and relationships.



Experiment No. 07

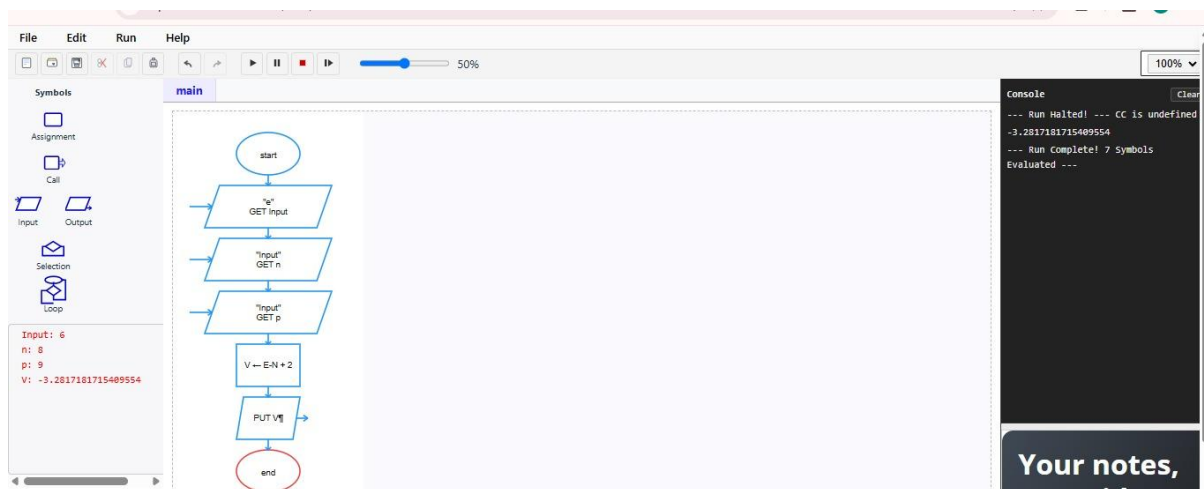
A college employs a large number of security personnel who are assigned duties at various locations across the campus. Each security person follows a duty schedule that includes date, duty start time, duty end time, and assigned location, ensuring that every campus location is covered. If a security person takes leave, it is recorded manually, and at the end of each month, salaries are calculated based on duties performed and leaves taken. Since manual handling of duty allocation, leave management, and salary calculation is time-consuming and complex, the college plans to develop an Online Security Management System to automate these activities. Develop Class Diagram, Use case Diagram and Activity Diagram to represent the workflow of the Online Security Management System.





Experiment No. 08

Cyclomatic Complexity in Software Testing is a testing metric used for measuring the complexity of a software program. It is a quantitative measure of independent paths in the source code of a software program. Cyclomatic complexity can be calculated by using control flow graphs or with respect to functions, modules, methods or classes within a software program. **Cyclomatic complexity for a flow graph G is $V(G)=E-N+2$.** Cyclomatic complexity is a software metric (measurement), used to indicate the complexity of a program, Cyclomatic complexity = $E - N + P$ where, E = number of edges in the flow graph, N = number of nodes in the flow graph, P = number of nodes that have exit points. Find Cyclomatic Complexity for a graph having number of edges as 17, number of nodes as 13 and number of predicate nodes in the flow graph as 5.



Experiment No. 09

Create a Scrum Project in Jira

Aim

To create a Scrum project in Jira, add backlog items, prioritize them, create a one-week sprint, move the backlog items into the sprint, and start the sprint to simulate the Scrum development process.

Software Requirements

Software	Version
Jira Cloud	Web Application
Web Browser	Google Chrome / Microsoft Edge
Internet Connection	Required

Theory

Scrum is an agile framework used in software development to manage and complete complex projects through short, time-boxed iterations known as sprints. It emphasizes iterative progress, collaboration, and continuous feedback among team members.

A Product Backlog is a prioritized list of features, tasks, or requirements that need to be completed during the course of a project. A Sprint is a fixed time period, typically one to four weeks, during which a selected set of backlog items is developed and delivered.

In Software Engineering, Jira supports the Scrum framework by allowing teams to create backlogs, plan sprints, and track task progress on a Sprint Board. This helps teams deliver working software incrementally while maintaining flexibility to adapt to changing requirements.

Procedure

Step 1: Create a Scrum Project

1. Log in to Jira Cloud.
2. Click Create Space.
3. Select Software Development.

4. Choose the Scrum template.

Enter the following project details:

Field	Value
Project Name	Online Shopping System
Management Type	Company-managed
Project Key	OSS

Click Create.

The screenshot shows the 'Name your space' form in Jira. On the left, the 'Space templates' sidebar is visible, with 'Software development' highlighted. Under 'CATEGORIES', 'Software development' is selected, and 'Scrum' is the chosen template, indicated by a red box and a blue arrow. The main form area is titled 'Name your space' and includes fields for 'Name' (filled with 'Online Food Delivery System1'), 'How your space is managed' (set to 'Company-managed'), and 'Space permissions' (set to 'ON'). A 'Next' button is at the bottom right.

Figure 1. Creating a Scrum Project.

Step 2: Create Product Backlog

Open the Backlog section and create the following backlog items.

Priority	Backlog Item
High	Create User Registration Page
High	Develop Login API
Medium	Design Product Listing Page
Medium	Implement Shopping Cart
Low	Test Checkout Module

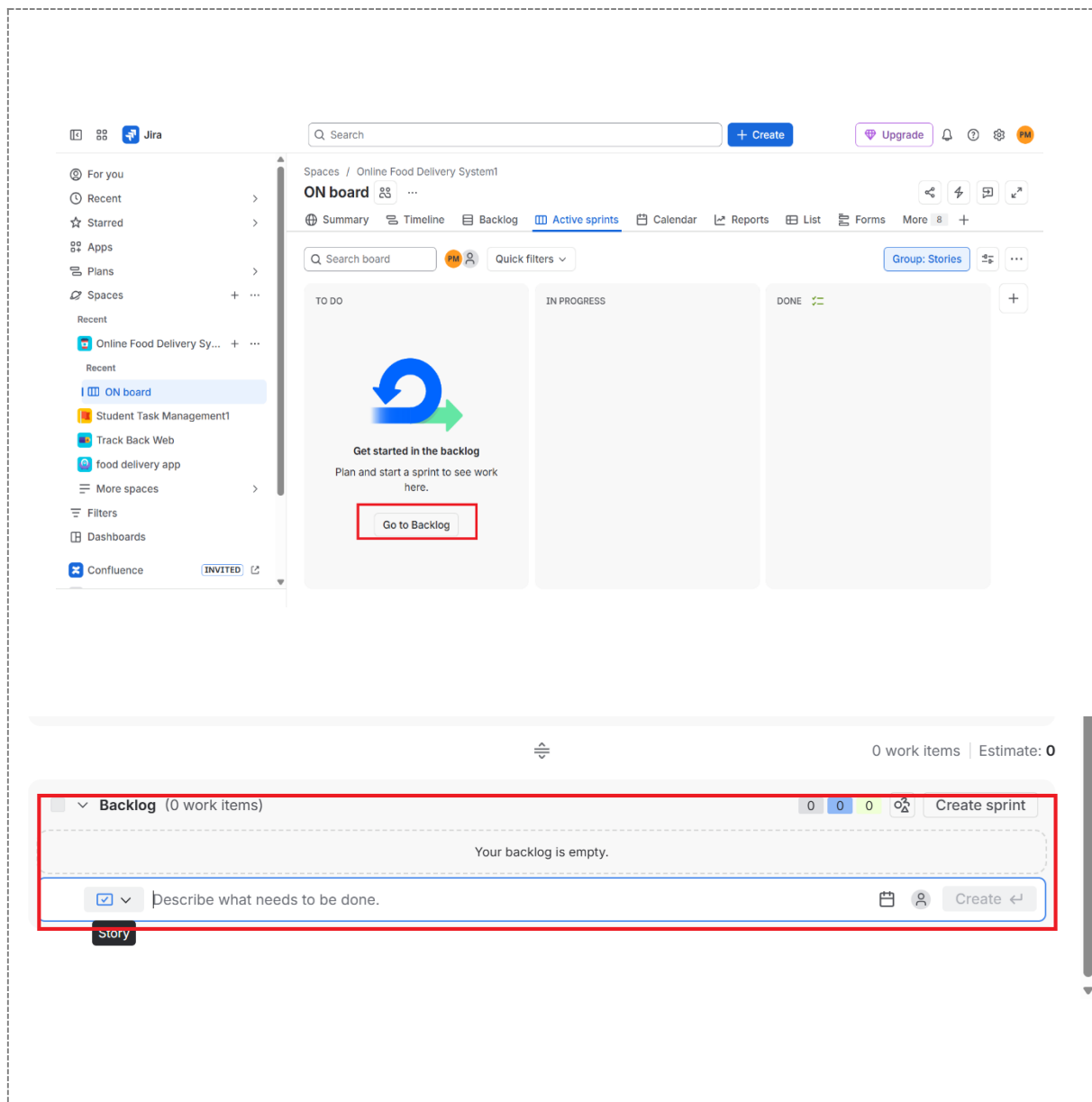


Figure 2. Creating Product Backlog Items.

Step 3: Prioritize the Backlog

Arrange the backlog in the following order:

1. Create User Registration Page
2. Develop Login API
3. Design Product Listing Page
4. Implement Shopping Cart
5. Test Checkout Module

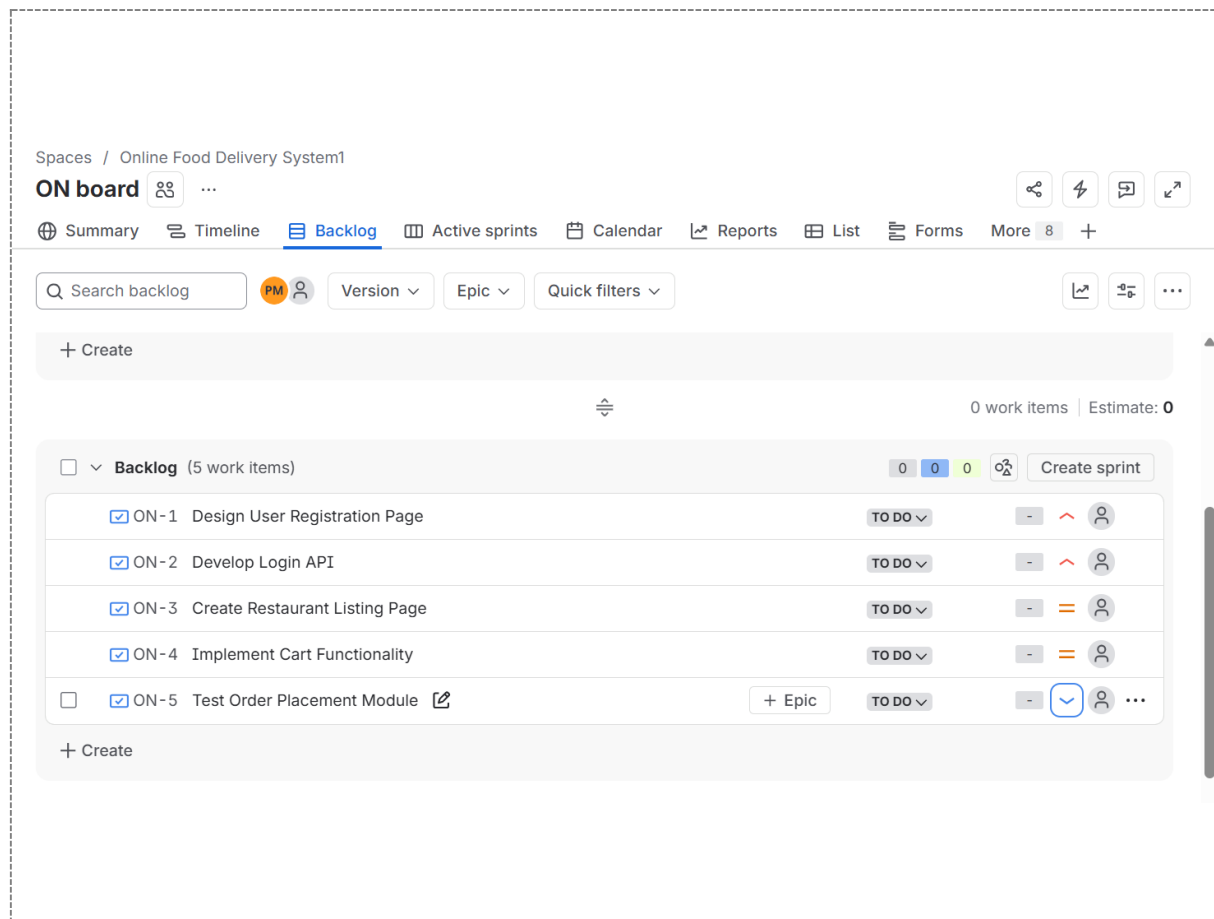


Figure 3. Prioritizing the Backlog.

Step 4: Create a Sprint

Click Create Sprint.

Enter the following details:

Field	Value
Sprint Name	Sprint 1
Duration	1 Week

Save the sprint.

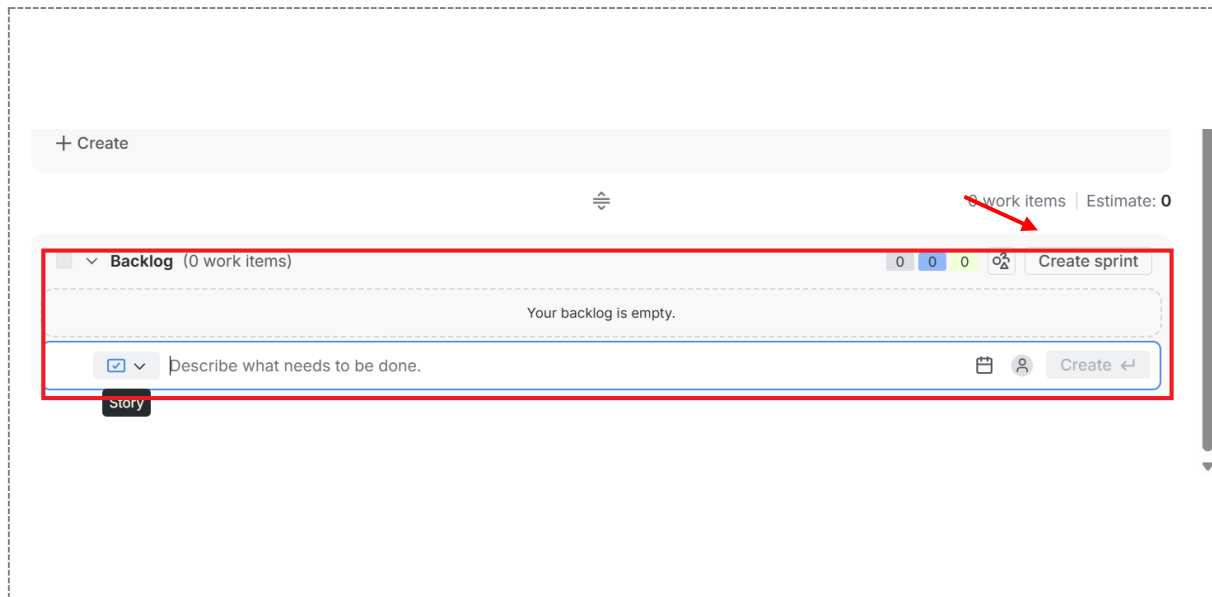


Figure 4. Creating Sprint 1.

Step 5: Add Backlog Items to Sprint

Move all five backlog items into Sprint 1.

Sprint Backlog:

- Create User Registration Page
- Develop Login API
- Design Product Listing Page
- Implement Shopping Cart
- Test Checkout Module

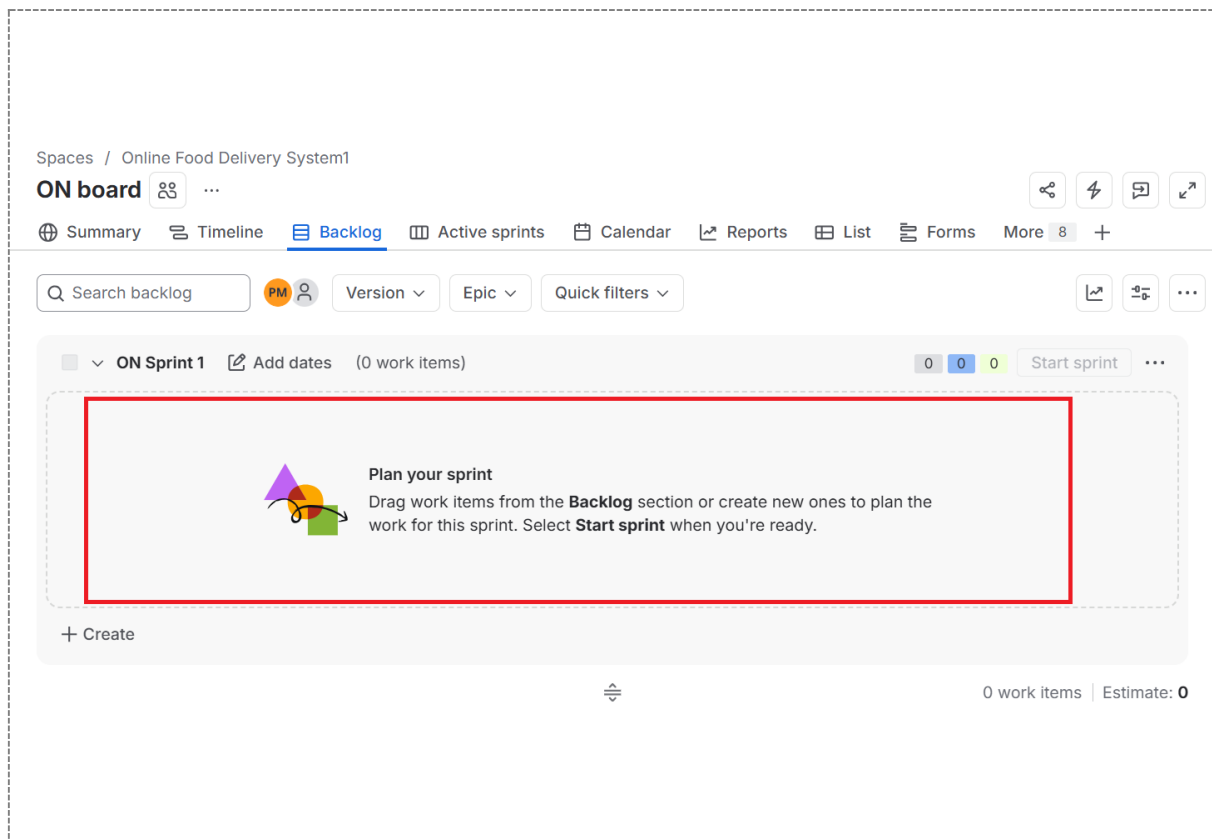


Figure 5. Adding Backlog Items to Sprint.

Step 6: Start the Sprint

1. Click Start Sprint.
2. Confirm the sprint duration is 1 Week.
3. Click Start.

The Sprint Board will display all tasks under the To Do column.

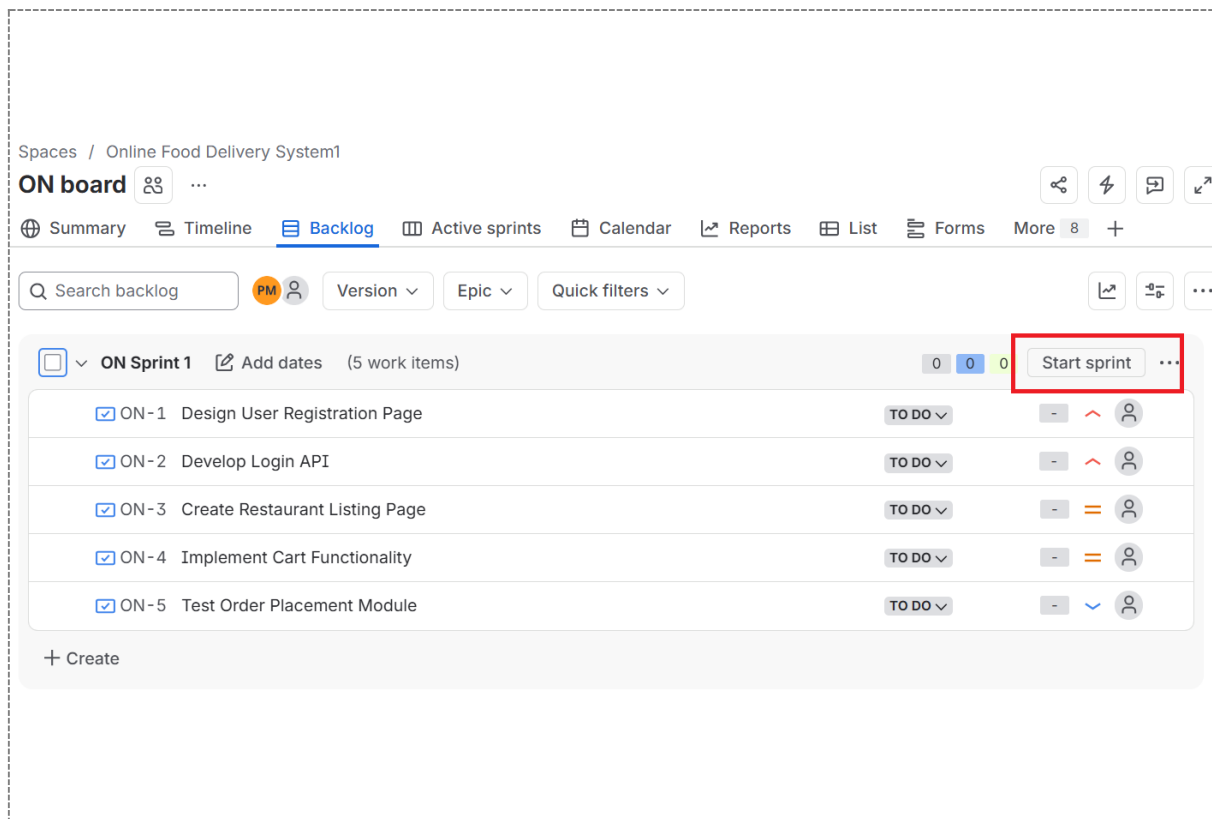


Figure 6. Starting the Sprint.

Step 7: Update Sprint Progress

Move the tasks to simulate development.

Task	Status
Create User Registration Page	Done
Develop Login API	Done
Design Product Listing Page	In Progress
Implement Shopping Cart	To Do
Test Checkout Module	To Do

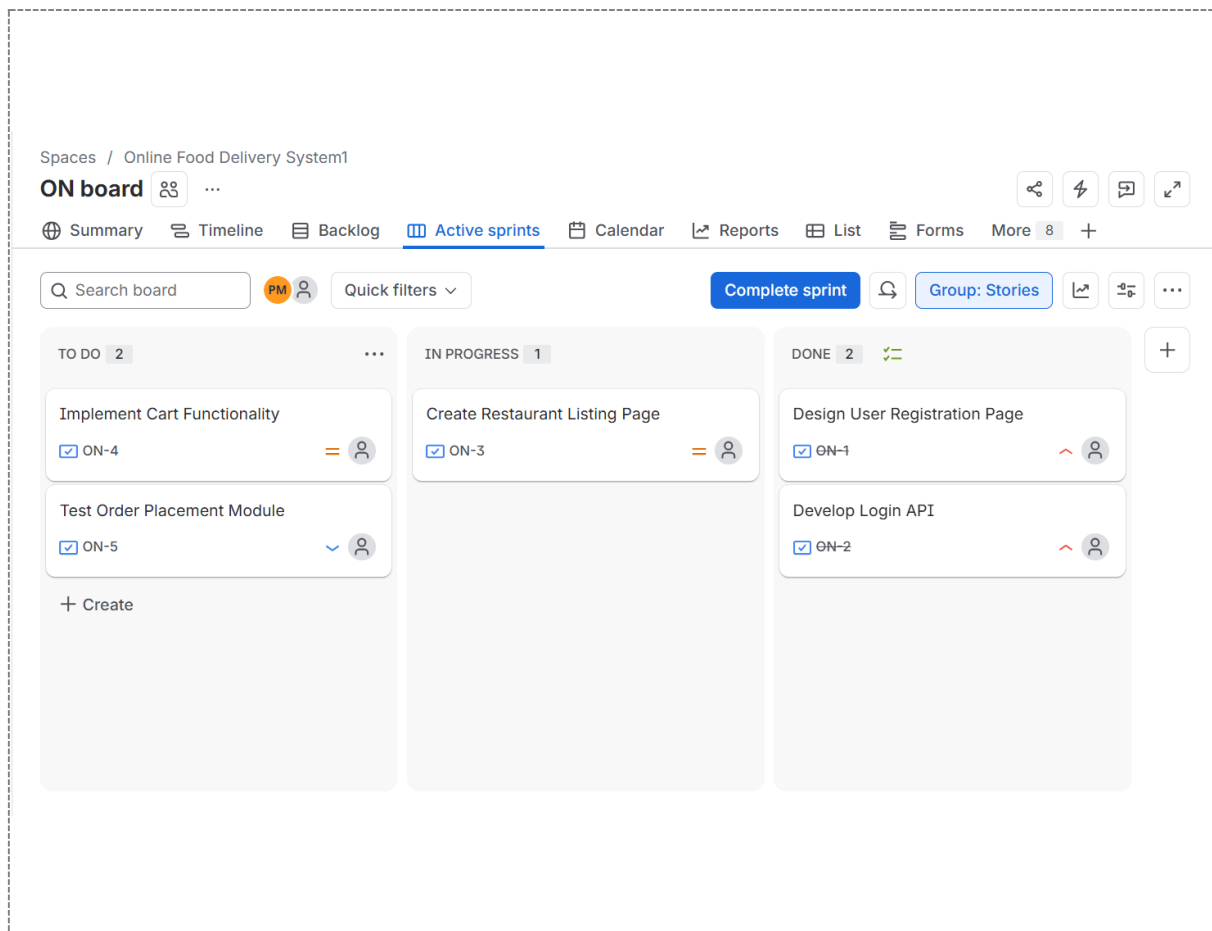


Figure 7. Updating Sprint Progress.

Step 8: Sprint Completion

The Sprint Board should display the following status:

Done

- Create User Registration Page
- Develop Login API

In Progress

- Design Product Listing Page

To Do

- Implement Shopping Cart
- Test Checkout Module

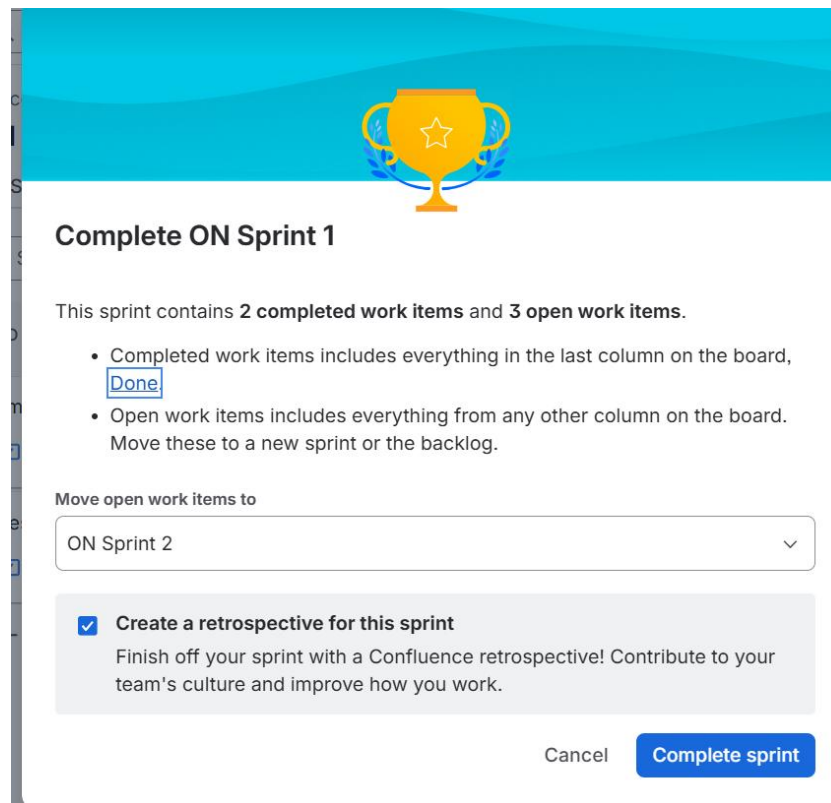


Figure 8. Sprint Completion Status.

Observation

Sprint Name	Total Tasks	Completed	In Progress	Pending
Sprint 1	5	2	1	2

Result

A Scrum project named Online Shopping System was successfully created in Jira. Five backlog items were added and prioritized, a one-week sprint was created, backlog items were assigned to the sprint, and the sprint was started successfully. The Sprint Board effectively demonstrated task progression using the Scrum workflow.

Experiment No. 10

Prioritize Software Requirements using the MoSCoW Method for a Library Management System

Aim

To analyze and categorize the functional requirements of a Library Management System using the MoSCoW Prioritization Method, based on user impact, feasibility, budget, and resource availability.

Software Requirements

Software	Version
Microsoft Excel / Google Sheets	Desktop Application

Theory

MoSCoW is a requirement prioritization technique used in software engineering to classify project requirements based on their importance and urgency. The name is derived from the first letters of its four categories: Must-Have, Should-Have, Could-Have, and Won't-Have, with the 'o's added simply to make the acronym pronounceable.

Procedure

Step 1: Identify the Requirements

Read the given requirements for the Library Management System carefully and understand the functionality expected in the software.

Step 2: Create a New Spreadsheet

Open Microsoft Excel or Google Sheets and create a new spreadsheet.

Step 3: Create Table Headings

Create a table with the following columns:

- Requirement

- MoSCoW Category
- Justification

Step 4: List the Requirements

Enter all the given functional requirements of the Library Management System into the Requirement column. The requirements include:

- Search books by title and author
- Online book reservation system
- Monthly borrowed books report
- Email notifications for overdue books
- QR code scanning
- Librarian dashboard
- Book reviews and ratings
- Chatbot assistance
- Mobile application
- Multi-language support

Step 5: Analyze Each Requirement

Evaluate every requirement based on the following criteria:

- Impact on users and stakeholders
- Feasibility considering available time
- Budget limitations
- Resource availability

Step 6: Categorize Using the MoSCoW Method

Assign each requirement to one of the following categories:

- Must-Have (M): Essential features required for the system to function.
- Should-Have (S): Important features that improve the system but are not critical.
- Could-Have (C): Desirable features that can be implemented if sufficient resources are available.
- Won't-Have (W): Features that are excluded from the current release and may be considered in future versions.

Step 7: Provide Justification

For each categorized requirement, write a brief justification explaining why it was assigned to that MoSCoW category based on its importance and feasibility.

Step 8: Save the Spreadsheet

Save the completed spreadsheet with an appropriate file name such as: Library_Management_System_MoSCoW_Prioritization.xlsx

Step 9: Verify the Data

Review the spreadsheet to ensure that:

- All requirements are included.
- Each requirement has an appropriate MoSCoW category.
- Every category has a clear and valid justification.
- The table is complete and properly formatted.

Observation

Requirement	MoSCoW Category	Justification
Search books by title and author	Must-Have	Core function required for the library system to be usable.
Librarian dashboard	Must-Have	Essential for librarians to manage books, users, and records.

Requirement	MoSCoW Category	Justification
Online book reservation system	Should-Have	Improves user convenience but the system can function without it initially.
Monthly borrowed books report	Should-Have	Useful for tracking usage, though not critical for day-to-day operation.
Email notifications for overdue books	Should-Have	Improves user experience and reduces overdue books, but not core functionality.
QR code scanning	Could-Have	Adds convenience for checkout but is not essential to core operations.
Book reviews and ratings	Could-Have	Enhances user engagement but has low impact on core library functions.
Mobile application	Could-Have	Desirable for accessibility but requires significant additional resources.
Chatbot assistance	Won't-Have	Low priority given current budget and time constraints; can be considered later.
Multi-language support	Won't-Have	Not essential for the initial release; deferred to a future version.

Result

The requirements of the Library Management System were successfully analyzed and categorized using the MoSCoW Prioritization Method. Each requirement was assigned to the appropriate priority level with a suitable justification based on user impact and project feasibility. The categorized requirements were documented in an Excel spreadsheet successfully.

A26		
A	B	C
1 Requirement	Category (MoSCoW)	Justification
2 Search books by title and author	Must-Have	Core functionality required for users to find books quickly.
3 Implement an online book reservation system	Must-Have	Essential for reserving unavailable books.
4 Generate monthly reports on borrowed books	Should-Have	Important for administrators but not critical for daily use.
5 Enable email notifications for overdue books	Should-Have	Improves communication and helps reduce overdue returns.
6 Add support for QR code scanning	Could-Have	Speeds borrowing and returning but manual process is still possible.
7 Create a user-friendly dashboard for librarians	Must-Have	Essential for efficient library management.
8 Allow users to review and rate books	Could-Have	Enhances user experience but is not essential.
9 Integrate a chatbot for user assistance	Could-Have	Helpful support feature but not necessary initially.
10 Develop a mobile app version of the system	Won't-Have	Deferred due to time and budget constraints.
11 Provide multi-language support	Won't-Have	Useful for future expansion but not required in the first release.
12		
13		

Experiment No. 11

Link Jira Tasks with Confluence for Library Management System

Aim

To integrate Jira work items with Confluence for the Library Management System project, enabling centralized task tracking, status monitoring, and visualization of sprint progress through embedded Jira issues and a progress indicator.

Software Requirements

Software	Version
Jira Cloud	Cloud Version
Confluence Cloud	Cloud Version
Google Chrome / Microsoft Edge	Latest Version

Theory

Confluence is a collaboration and documentation tool provided by Atlassian, while Jira is a project management and issue-tracking application. Integrating Jira with Confluence enables project teams to embed Jira work items directly into Confluence pages, providing real-time visibility of project progress.

The Jira macro displays issue details such as status, assignee, and priority, while a progress indicator helps monitor sprint completion efficiently. This integration improves communication, collaboration, and project documentation.

Procedure

Step 1: Create a Library Management System Project in Jira

1. Log in to Jira Cloud.
2. Click Create Space → Software Development.
3. Select the Scrum template.

Enter the following details:

- Project Name: Library Management System
 - Project Key: LMS
4. Click Create.

Step 2: Create Jira Work Items

Open the Backlog and create the following Jira work items.

Issue ID	Summary	Status
LMS-1	Develop Book Search Functionality	Done
LMS-2	Create User Login Page	In Progress
LMS-3	Implement Book Borrowing Module	To Do

Issue ID	Summary	Status
LMS-4	Design Librarian Dashboard	In Progress
LMS-5	Test Book Return Module	Done

Arrange the work items so that their statuses appear under To Do, In Progress, and Done.

Spaces / Library Management System

LMS board  ...

Summary Timeline Backlog **Active sprints** Calendar Reports List Forms More 8 +

Q Search board  Quick filters  **Complete sprint**  **Group: Story**   ...

TO DO 2 ...

Design Librarian Dashboard

☒ LMS-4 

Test Book Return Module

☒ LMS-5 

+ Create

IN PROGRESS 1

Implement Book Borrowing Module

☒ LMS-3 

DONE 2 

Develop Book Search Functionality

☒ LMS-1 

Create User Login Page

☒ LMS-2 

Step 3: Open Confluence

5. Open your Atlassian Workspace.
6. Select Confluence.
7. Click Create to create a new page.

Step 4: Create a New Confluence Page

Create a page with the title: "Library Management System Project Overview"

Step 5: Add the Project Description

Below the page title, enter the following description:

"The Library Management System project is designed to automate library operations such as user login, book searching, borrowing, and returning books. This page provides an overview of the project tasks and tracks their current development status by integrating Jira work items into Confluence."

Step 6: Add a Heading

Create the heading: "Development Tasks"

Step 7: Insert Jira Work Items

8. Place the cursor below the heading.
9. Type /jira.
10. Select Jira Work Items.

11. Search for the Library Management System project.

Select the following Jira issues:

- LMS-1 – Develop Book Search Functionality
- LMS-2 – Create User Login Page
- LMS-3 – Implement Book Borrowing Module
- LMS-4 – Design Librarian Dashboard
- LMS-5 – Test Book Return Module

12. Click Insert.

Verify that the embedded Jira work items display their current status (To Do, In Progress, and Done).

The screenshot shows a Confluence page titled "Library Management System Project Overview-Pramith" by Pramith M. The page content includes a description of the project and a table of development tasks.

Library Management System Project Overview-Pramith
By Pramith M

The Library Management System is developed to simplify the management of library resources. The project enables users to search books, reserve books online, borrow and return books efficiently, and assists librarians in managing the library through a centralized system. This page provides an overview of the project tasks and their current development status.

Development Tasks

Type	Key	Summary	Assignee	Priority	Status	Updated
Task	LMS-5	Test Book Return Module		Medium	TO DO	Jul 27, 2026, 09:37

Step 8: Publish the Page

13. Review the page contents.

Ensure that:

- The page title is correct.
- The project description is present.
- Five Jira work items are embedded.
- The task status is displayed correctly.
- The progress indicator is visible.

14. Click Publish.

Step 9: Verify the Page

Review the published Confluence page to ensure that:

- The page title is displayed correctly.
- All five Jira work items are embedded successfully.

- Task statuses (To Do, In Progress, and Done) are visible.
- The progress indicator accurately reflects the sprint completion.
- The page is properly formatted.

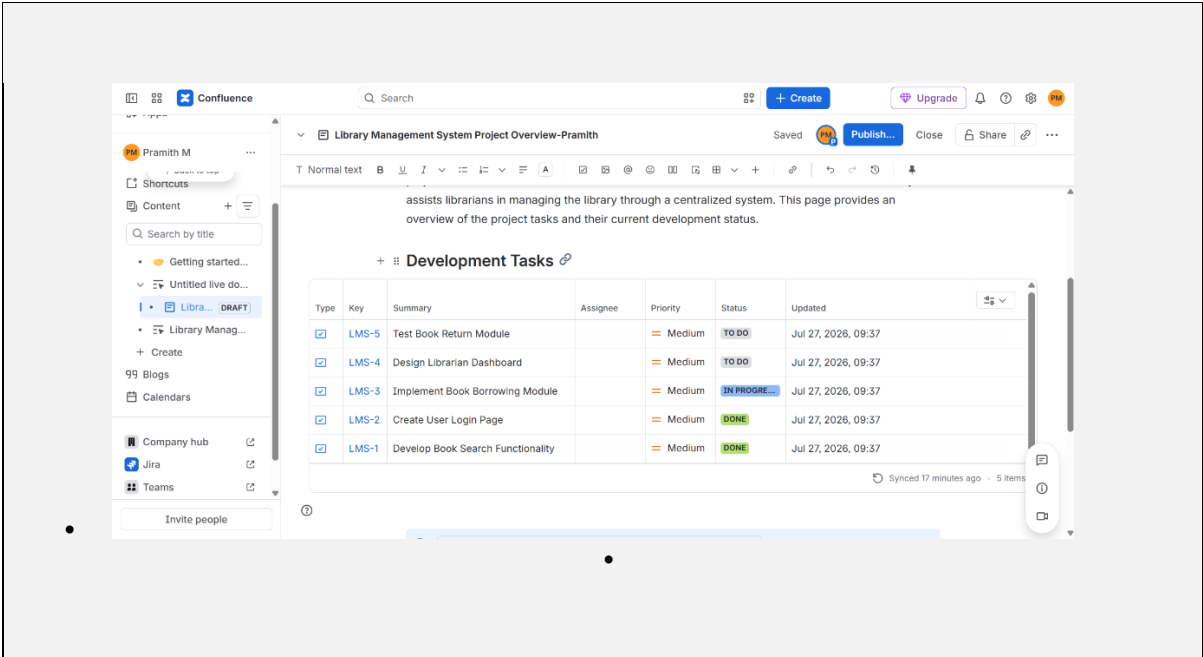


Figure 1. Library Management System Project Overview Page.

Observation

Embedded Jira Task	Status
Develop Book Search Functionality	Done
Create User Login Page	In Progress
Implement Book Borrowing Module	To Do
Design Librarian Dashboard	In Progress
Test Book Return Module	Done

Result

The Library Management System Project Overview page was successfully created in Confluence. Five Jira work items were embedded using the Jira macro, and their current status was displayed successfully. A progress indicator was added to visualize sprint completion, enabling efficient task tracking, collaboration, and progress monitoring throughout the project.

Experiment No. 13

You are tasked with developing an Online Learning Platform. The platform should include the following functionalities:

Course Enrollment and Registration
Video Lecture Streaming
Interactive Quizzes and Assignments
Progress Tracking Dashboard
Peer-to-Peer Discussion Forums
Certificate Generation

Use Jira to categorize and prioritize these requirements using MoSCoW and Kano techniques

Aim

To categorize and prioritize the functional requirements of an Online Learning Platform using the **MoSCoW** and **Kano** prioritization techniques, implemented through **Jira** software.

Software Required

Jira (Cloud version — jira.atlassian.com / your organization's Jira instance)

Requirements to be Prioritized

1. Course Enrollment and Registration
2. Video Lecture Streaming
3. Interactive Quizzes and Assignments
4. Progress Tracking Dashboard
5. Peer-to-Peer Discussion Forums
6. Certificate Generation

Part A: Theory

MoSCoW Technique

MoSCoW classifies requirements into four categories:

- **Must Have** — critical, non-negotiable for launch
- **Should Have** — important but not vital; can be delayed if needed
- **Could Have** — desirable but not necessary; nice-to-have

- **Won't Have (this time)** — out of scope for the current release

Kano Model

Kano classifies requirements based on customer satisfaction impact:

- **Basic (Must-be)** — expected by users; absence causes dissatisfaction
 - **Performance** — satisfaction increases proportionally with how well it's implemented
 - **Excitement (Attractive)** — unexpected features that delight users
 - **Indifferent** — features users don't care about either way
-

Part B: Procedure in Jira

Step 1: Create a Jira Project

1. Log in to Jira at **your-domain.atlassian.net**.
2. Click "**Create Project**" on the left sidebar.
3. Select template: **Scrum** or **Kanban** (Scrum is preferred for backlog prioritization).
4. Name the project: "**Online Learning Platform**".
5. Set the project key (e.g., **OLP**) and click **Create**.

Step 2: Create an Epic

1. Go to the **Backlog** view.
2. Click "**Create**" (top bar) → set Issue Type to **Epic**.
3. Name the epic: "**Online Learning Platform - Core Features**".
4. Click **Create**.

Step 3: Create Issues for Each Requirement

1. In the Backlog, click "**Create issue**".

2. Set **Issue Type** to **Story**.
3. Enter the **Summary** field with each requirement, one at a time:
 - Course Enrollment and Registration
 - Video Lecture Streaming
 - Interactive Quizzes and Assignments
 - Progress Tracking Dashboard
 - Peer-to-Peer Discussion Forums
 - Certificate Generation
4. Link each Story to the Epic created in Step 2 (use the **Epic Link** field).
5. Click **Create** after each entry — this results in 6 stories under the epic.

Step 4: Set Up Custom Labels for MoSCoW

1. Open each issue (Story) one at a time.
2. On the right panel, find the **Labels** field.
3. Click **Add label** and type one of the following (create new labels if they don't exist):
 - must-have
 - should-have
 - could-have
 - wont-have
4. Assign labels based on the analysis below:

Requirement	MoSCoW Category	Justification
Course Enrollment and Registration	Must Have	Core entry point; platform cannot function without it
Video Lecture Streaming	Must Have	Primary content delivery mechanism
Interactive Quizzes and Assignments	Should Have	Important for learning outcomes, not blocking launch
Progress Tracking Dashboard	Should Have	Enhances user experience, not critical to MVP
Peer-to-Peer Discussion Forums	Could Have	Adds engagement but platform works without it
Certificate Generation	Could Have	Valuable add-on, can be released in a later version

Step 5: Set Up Custom Field for Kano Classification

1. Go to **Project Settings** → **Fields** (Jira Admin access required).
2. Click "**Add Field**" → choose **Select List (single choice)**.
3. Name the field: "**Kano Category**".
4. Add options: Basic, Performance, Excitement, Indifferent.
5. Attach this field to your project's issue screen.
6. Go back to each Story and set its **Kano Category** value:

Requirement	Kano Category	Justification
Course Enrollment and Registration	Basic	Expected baseline; its absence causes major dissatisfaction
Video Lecture Streaming	Basic	Core expected functionality of any learning platform
Interactive Quizzes and Assignments	Performance	More/better quizzes = higher satisfaction
Progress Tracking Dashboard	Performance	Better tracking = more perceived value
Peer-to-Peer Discussion Forums	Excitement	Not expected by default; delights users when present
Certificate Generation	Excitement	Adds unexpected value (credibility, motivation)

Step 6: Prioritize the Backlog

1. Go to the **Backlog** view.
2. Drag and drop issues to reorder them — highest priority (Must Have + Basic) at the top.
3. Use the **Priority** field (top-right of each issue) to also set: **Highest, High, Medium, Low**, aligned with the MoSCoW ranking.

Step 7: Create a Board View for Visualization

1. Go to **Project Settings → Board → Columns**.
2. Set up columns: **To Do, In Progress, In Review, Done**.
3. Use **Quick Filters** (Board settings) to filter issues by label, e.g., filter by must-have to view only critical requirements.

Step 8: Generate Reports

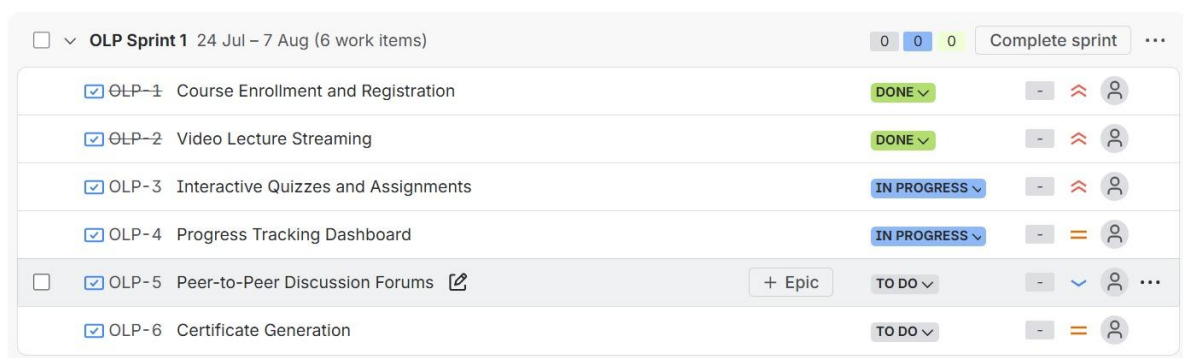
1. Go to the **Reports** section in the left sidebar.
2. Open **Sprint Report** or **Burndown Chart** to visualize how prioritized items are progressing (useful once development sprints begin).

Step 9: Document and Export

1. Go to **Backlog**, select all issues.
2. Export the prioritized list: click "..." (More actions) → **Export** → choose **CSV** or **Excel**.
3. Save this exported file as supporting evidence/screenshot for your lab record.

Result

The six functional requirements of the Online Learning Platform were successfully categorized and prioritized using the MoSCoW technique (Must/Should/Could/Won't) and the Kano model (Basic/Performance/Excitement/Indifferent) within Jira, and the backlog was organized accordingly for sprint planning.



OLP board



Summary Timeline Backlog **Active sprints** Calendar Reports List Forms More 8 +

Q Search board PM Label Quick filters Complete sprint Group: Stories

TO DO 2

Peer-to-Peer Discussion Forums

☒ OLP-5

Certificate Generation

☒ OLP-6

+ Create

IN PROGRESS 2

Interactive Quizzes and Assignments

☒ OLP-3

Progress Tracking Dashboard

☒ OLP-4

DONE 2

Course Enrollment and Registration

☒ OLP-1

Video Lecture Streaming

☒ OLP-2